

Assignment-2

(Collections, functions and Modules)

[Accessing List]

1.Understanding how to create and access elements in a list.

➤ Introduction to Lists

A list is a collection data type in Python used to store multiple values in a single variable. Lists help organize related data together and make it easier to manage large amounts of information.

Lists are:

- Ordered
- Mutable (changeable)
- Able to store duplicate values
- Capable of storing different data types

Lists are represented using square brackets [].

➤ Creating a List

Creating a list means defining and storing elements inside it. Elements are separated by commas and enclosed within square brackets.

A list may contain:

- Numbers
- Strings
- Boolean values
- Mixed data types
- Other lists

When a list is created, Python stores each element in a specific position.

➤ Accessing Elements in a List

Accessing elements means retrieving values stored inside the list.

Elements are accessed using their index numbers.

Python uses the index to identify which element should be returned.

For example:

- Index 0 accesses the first element

- Index 1 accesses the second element
- Index 2 accesses the third element

2.Indexing in lists (positive and negative indexing).

Indexing is the process of accessing individual elements stored in a list using their position numbers. Every element in a list is assigned a unique index value that helps Python identify and retrieve that element.

➤ Positive Indexing

Positive indexing accesses elements from the beginning of the list toward the end.

The counting starts from left to right.

Characteristics of Positive Indexing

- Starts from 0
- Moves forward through the list
- Used to access elements from the start

➤ Negative Indexing

Negative indexing accesses elements from the end of the list toward the beginning.

The counting starts from right to left.

Characteristics of Negative Indexing

- Starts from -1
- Used to access elements from the end
- Helpful when the list length is unknown

3.Slicing a list: accessing a range of elements.

Slicing is a method used to access multiple elements from a list at the same time. It allows a portion of the list to be retrieved instead of accessing one element individually.

In slicing, a range of indexes is specified:

- The starting index is included.
- The ending index is excluded.

Slicing can be performed using positive or negative indexes.

It is useful for:

- Extracting parts of a list
- Working with subsets of data
- Simplifying list operations

[List Operations]

4. Common list operations: concatenation, repetition, membership

➤ Concatenation

Concatenation is the process of combining two or more lists into a single list. It joins all elements together in sequence and creates a new list.

This operation is useful when multiple lists need to be merged into one collection.

➤ Repetition

Repetition is used to repeat the elements of a list multiple times. It creates a new list where the original elements appear repeatedly according to the specified number.

This operation is useful for creating repeated patterns or duplicate data quickly.

➤ Membership

Membership is used to check whether a particular element exists in a list or not.

It helps determine:

- If an element is present
- If an element is absent

Membership operations are commonly used in searching and condition checking within programs.

5. Understanding list methods like `append()`, `insert()`, `remove()`, `pop()`.

➤ `append()`

The `append()` method is used to add a new element at the end of a list. It increases the size of the list by adding the element after the last existing item.

This method is useful when new data needs to be added continuously.

➤ `insert()`

The `insert()` method is used to add an element at a specific position in a list. Existing elements are shifted to make space for the new element.

This method is useful when data must be placed at a particular index.

➤ `remove()`

The `remove()` method is used to delete a specific element from a list. It removes the first occurrence of the specified value.

This method is useful for eliminating unwanted elements from a list.

➤ `pop()`

The `pop()` method is used to remove an element from a list using its index position. If no index is specified, it removes the last element by default.

It is useful when an element needs to be removed and returned for further use.

[Working with Lists]

6. Iterating over a list using loops

Iteration means accessing each element of a list one by one using loops. It allows programmers to process all elements in a list without accessing them individually.

Loops automatically repeat a set of instructions for every element present in the list.

The most commonly used loop for list iteration is the for loop. During iteration, each element is accessed in sequence from the beginning to the end of the list.

Iteration is useful for:

- Displaying list elements
- Performing calculations
- Searching for values
- Modifying data
- Processing large collections of information

Using loops for iteration makes programs shorter, faster, and more efficient when working with lists.

7. Sorting and reversing a list using `sort()`, `sorted()`, and `reverse()`.

➤ `sort()`

The `sort()` method is used to arrange the elements of a list in a specific order, usually ascending by default. It modifies the original list directly.

Sorting helps organize data in a meaningful and systematic way, making it easier to search and analyze information.

➤ `sorted()`

The `sorted()` function is also used to arrange elements in order, but unlike `sort()`, it creates and returns a new sorted list without changing the original list.

This is useful when the original data must remain unchanged while working with a sorted version.

➤ `reverse()`

The `reverse()` method is used to reverse the order of elements in a list. The first element becomes the last, and the last element becomes the first.

Reversing is useful when data needs to be displayed or processed in the opposite order.

8. Basic list manipulations: addition, deletion, updating, and slicing.

➤ Addition

Addition in a list refers to adding new elements to the list. Elements can be added at the end or at a specific position. This operation increases the size of the list and helps store new data dynamically.

Addition is useful when new information needs to be included in an existing list.

➤ **Deletion**

Deletion is the process of removing elements from a list. Elements can be removed by specifying either the value or the position of the element.

Deletion helps eliminate unwanted or unnecessary data from the list.

➤ **Updating**

Updating means changing the value of an existing element in a list. Since lists are mutable, elements can be modified after the list is created.

Updating is useful when stored data needs correction or modification.

➤ **Slicing**

Slicing is used to access a range of elements from a list. It extracts a portion of the list based on specified index positions.

Slicing helps work with subsets of data efficiently without changing the original list.

[Tuple]

9. Introduction to tuples, immutability.

A tuple is a collection data type in Python used to store multiple elements in a single variable. Tuples are similar to lists because they can contain different types of data and maintain the order of elements.

Tuples are created using parentheses ().

Tuples can store:

- Numbers
- Strings
- Boolean values
- Mixed data types
- Nested collections

Each element in a tuple has an index position, which allows elements to be accessed easily.

➤ **Immutability of Tuples**

The most important characteristic of a tuple is **immutability**.

Immutability means that once a tuple is created:

- Elements cannot be changed

- New elements cannot be added
- Existing elements cannot be removed

The contents of a tuple remain fixed throughout the program.

Because of immutability:

- Tuples are more secure for fixed data
- They use memory efficiently
- They are faster than lists in some operations

➤ **Uses of Tuples**

Tuples are commonly used when:

- Data should not be modified
- Fixed collections are required
- Protecting data from accidental changes
- Returning multiple values from functions

10. Creating and accessing elements in a tuple.

A tuple is created by placing elements inside parentheses (), separated by commas. A tuple can contain multiple values of the same or different data types.

Tuples are ordered collections, meaning the elements are stored in a fixed sequence.

A tuple may contain:

- Numbers
- Strings
- Boolean values
- Mixed data types
- Nested tuples

Once created, the elements of a tuple cannot be changed because tuples are immutable.

➤ **Accessing Elements in a Tuple**

Elements in a tuple are accessed using index numbers. Each element has a specific position that starts from 0.

Positive Indexing

- Starts from the beginning of the tuple
- First element has index 0
- Moves from left to right

Negative Indexing

- Starts from the end of the tuple
- Last element has index -1
- Moves from right to left

Tuples also support slicing, which allows accessing a range of elements from the tuple.

11. Basic operations with tuples: concatenation, repetition, membership.

➤ **Concatenation**

Concatenation is the process of combining two or more tuples into a single tuple. It joins all elements together in sequence and creates a new tuple.

Since tuples are immutable, concatenation does not modify the original tuples. Instead, a new tuple is created containing all elements.

Concatenation is useful for merging related data collections.

➤ **Repetition**

Repetition is used to repeat the elements of a tuple multiple times. It creates a new tuple where the original elements appear repeatedly according to the specified number.

This operation is useful for creating repeated patterns or duplicating data quickly.

➤ **Membership**

Membership is used to check whether a particular element exists in a tuple or not.

It helps determine:

- If an element is present
- If an element is absent

Membership operations are commonly used in searching and condition checking.

[Accessing Tuples]

12. Accessing tuple elements using positive and negative indexing.

Elements in a tuple are accessed using index numbers. Each element in a tuple has a specific position, which allows direct access to the stored data.

Since tuples are ordered collections, indexing helps retrieve elements efficiently.

➤ **Positive Indexing**

Positive indexing accesses elements from the beginning of the tuple.

Characteristics

- Starts from 0

- Moves from left to right
- First element has index 0

➤ **Negative Indexing**

Negative indexing accesses elements from the end of the tuple.

Characteristics

- Starts from -1
- Moves from right to left
- Last element has index -1

13. Slicing a tuple to access ranges of elements.

Slicing is a method used to access multiple elements from a tuple at the same time. It allows a portion of the tuple to be retrieved instead of accessing one element individually.

In slicing, a range of indexes is specified:

- The starting index is included
- The ending index is excluded

Tuples support both positive and negative indexing for slicing.

Slicing is useful for:

- Extracting specific parts of a tuple
- Working with subsets of data
- Accessing ranges of elements efficiently

Since tuples are immutable, slicing does not modify the original tuple. Instead, it creates a new tuple containing the selected elements.

[Dictionaries]

14. Introduction to dictionaries: key-value pairs.

A dictionary is a built-in data structure in Python used to store data in the form of **key-value pairs**. It is used when data needs to be organized and accessed using unique identifiers called keys.

Dictionaries are created using curly braces {}.

In a dictionary:

- A **key** acts as an identifier or label
- A **value** is the data associated with that key

Each key is connected to a specific value, forming a key-value pair.

Example conceptually:

```
{  
  "name": "Aayushi",  
  "age": 21  
}
```

Here:

- "name" and "age" are keys
- "Aayushi" and 21 are values

➤ Characteristics of Dictionaries

- Key-Value Structure

Data is stored as pairs of keys and values, making information easy to access.

- Unique Keys

Keys in a dictionary must be unique. Duplicate keys are not allowed.

- Mutable

Dictionaries are mutable, meaning values can be added, updated, or removed after creation.

- Unordered Collection

Dictionaries store data based on keys rather than index positions.

15. Accessing, adding, updating, and deleting dictionary elements.

➤ Accessing Dictionary Elements

Dictionary elements are accessed using their keys. Each key is associated with a specific value, and the key is used to retrieve that value.

Accessing through keys allows fast and efficient retrieval of data from the dictionary.

➤ Adding Dictionary Elements

New elements can be added to a dictionary by assigning a new key with a value. When a new key-value pair is added, the size of the dictionary increases.

Adding elements is useful when new information needs to be stored dynamically.

➤ Updating Dictionary Elements

Updating means changing the value associated with an existing key. Since dictionaries are mutable, values can be modified after the dictionary is created.

Updating is useful for correcting or modifying stored information.

➤ Deleting Dictionary Elements

Dictionary elements can be removed by specifying their keys. Deletion removes both the key and its associated value from the dictionary.

Deleting elements is useful for removing unwanted or unnecessary data.

16. Dictionary methods like keys(), values(), and items().

➤ keys()

The keys() method is used to access all the keys present in a dictionary. It returns a collection of keys stored in the dictionary.

This method is useful when only the identifiers or labels of the dictionary are needed.

➤ values()

The values() method is used to access all the values stored in a dictionary. It returns a collection of all values associated with the keys.

This method is useful when only the stored data needs to be processed or displayed.

➤ items()

The items() method is used to access both keys and values together as pairs. It returns all dictionary elements in the form of key-value pairs.

This method is useful when both keys and their corresponding values are required during processing or iteration.

[Working with Dictionaries]

17. Iterating over a dictionary using loops

Iteration over a dictionary means accessing its elements one by one using loops. Since dictionaries store data in the form of key-value pairs, iteration can be performed over keys, values, or both together.

Loops allow all elements in the dictionary to be processed automatically without accessing each item individually.

The most commonly used loop for dictionary iteration is the for loop.

➤ Types of Dictionary Iteration

- Iterating Through Keys

This accesses all keys present in the dictionary one by one.

- Iterating Through Values

This accesses all values stored in the dictionary.

- Iterating Through Key-Value Pairs

This accesses both keys and their corresponding values together during iteration.

18. Merging two lists into a dictionary using loops or zip().

Merging two lists into a dictionary means combining one list as keys and another list as values to create key-value pairs.

In this process:

- The first list usually contains keys
- The second list contains corresponding values

The elements at the same positions in both lists are combined together.

➤ Using Loops

A loop can be used to iterate through both lists and manually create key-value pairs in the dictionary.

This method provides more control over the merging process and allows additional operations during iteration.

➤ Using zip()

The zip() function combines elements from two lists position by position. It pairs corresponding elements together, making dictionary creation simpler and faster.

zip() is useful for:

- Reducing code complexity
- Efficiently combining lists
- Creating dictionaries quickly

19. Counting occurrences of characters in a string using dictionaries.

A dictionary is a collection that stores data in the form of **key-value pairs**.

In character counting:

- The **key** represents the character.
- The **value** represents how many times that character appears in the string.

The process works by reading each character of the string one by one.

If a character appears for the first time, it is added to the dictionary with a count of 1.

If the character already exists in the dictionary, its count is increased.

This method helps in easily tracking the frequency of every character in a string.

For example, in a string containing repeated letters, the dictionary keeps updating the count whenever the same character is found again.

Using dictionaries for character counting is efficient because dictionaries allow quick searching and updating of data.

➤ Uses of Character Counting

- Text analysis
- Finding repeated characters
- Word and letter frequency checking
- Data processing

- Anagram checking

➤ **Advantages**

- Fast and efficient
- Easy to organize data
- Simple way to store frequencies
- Works well for large strings

[Functions]

20. Defining functions in Python

A function in Python is a block of organized and reusable code designed to perform a specific task. Functions help divide a program into smaller parts, making the code easier to understand, manage, and reuse.

Functions are defined using the `def` keyword followed by the function name and parentheses. The statements inside the function are executed only when the function is called.

A function can:

- Perform a particular operation
- Accept input values called parameters
- Return a result after execution

Functions improve program structure by reducing repetition of code and increasing readability.

➤ **Main Parts of a Function**

- Function Name – Identifies the function
- Parameters – Values passed into the function
- Function Body – Statements that define the task
- Return Statement – Sends a result back to the caller

➤ **Benefits of Using Functions**

- Code reusability
- Better organization of programs
- Easier debugging and maintenance
- Reduces duplication of code
- Makes programs more modular

➤ **Types of Functions in Python**

- Built-in functions

- User-defined functions
- Applications of Functions
- Performing calculations
 - Processing data
 - Organizing large programs
 - Reusing logic in multiple places

21. Different types of functions: with/without parameters, with/without return values.

Functions in Python can be classified based on:

1. Whether they accept parameters
2. Whether they return values

According to these features, functions are divided into four main types.

➤ Function Without Parameters and Without Return Value

This type of function:

- Does not take any input from the user
 - Does not return any value after execution
- It simply performs a task defined inside the function.

Characteristics

- Simplest type of function
- Used for displaying messages or performing fixed operations
- No data is passed to the function

➤ Function With Parameters and Without Return Value

This type of function:

- Accepts input values called parameters
- Performs operations using those values
- Does not return any result

Characteristics

- Input is provided from outside the function
- Useful when the same operation is needed with different values
- Results are usually displayed directly inside the function

➤ Function Without Parameters and With Return Value

This type of function:

- Does not accept any input values
- Performs operations internally
- Returns a value after execution

Characteristics

- Useful when data is generated inside the function
- The returned value can be stored or used elsewhere in the program

22. Anonymous functions (lambda functions).

An anonymous function is a function that is defined without a name.

In Python, anonymous functions are created using the lambda keyword, so they are also called **lambda functions**.

Lambda functions are small, simple functions that are usually used for short-term operations where creating a normal function is unnecessary.

➤ Characteristics of Lambda Functions

- Defined using the lambda keyword
- Do not require a function name
- Can have any number of parameters
- Contain only one expression
- Automatically return the result of the expression
- Mostly used for short and simple tasks

➤ Purpose of Lambda Functions

Lambda functions are used when:

- A function is needed only temporarily
- Small operations need to be performed quickly
- Writing a full function definition is unnecessary
- Functions are passed as arguments to other functions

[Modules]

23. Introduction to Python modules and importing modules.

A module in Python is a file that contains Python code such as functions, variables, classes, and statements. Modules help organize programs into separate files, making the code easier to manage and reuse.

Instead of writing all code in a single file, programmers divide related code into modules. This improves readability, maintenance, and reusability of programs.

Python provides:

- Built-in modules – Already available in Python
- User-defined modules – Created by programmers

➤ Purpose of Modules

Modules are used to:

- Organize large programs
- Reuse code in different programs
- Reduce code duplication
- Make programs easier to maintain
- Improve readability

➤ Importing Modules

To use the contents of a module in a program, the module must be imported.

Importing means loading the module so its functions, variables, or classes can be accessed in another Python file.

Python provides different ways to import modules depending on the requirement.

24. Standard library modules: math, random.

Python provides many built-in modules in its standard library. These modules contain predefined functions and tools that help perform common programming tasks without writing code from scratch.

Two commonly used standard library modules are:

- math module
- random module

➤ Math Module

The math module provides mathematical functions and constants used for performing advanced mathematical operations.

It is useful in calculations involving:

- Arithmetic operations
- Trigonometry
- Logarithms
- Powers and roots
- Mathematical constants

The module helps programmers perform complex mathematical computations easily and accurately.

Features of the Math Module

- Supports advanced mathematical calculations
- Provides predefined mathematical functions
- Improves accuracy in calculations
- Useful in scientific and engineering programs

➤ **Random Module**

The random module is used to generate random values.

Random values are numbers or selections produced without a fixed pattern. This module is useful in applications where unpredictability is needed.

Features of the Random Module

- Generates random numbers
- Selects random items
- Produces random sequences
- Supports simulations and games

25. Creating custom modules.

A custom module is a module created by the programmer to store reusable Python code such as functions, variables, and classes.

Custom modules help organize programs into separate files so the same code can be reused in multiple programs without rewriting it.

In Python, any file with the .py extension can act as a module.

➤ **Purpose of Custom Modules**

Custom modules are created to:

- Reuse code in different programs
- Organize large projects
- Reduce code duplication
- Improve readability and maintenance
- Support modular programming

➤ **Process of Creating a Custom Module**

Creating a custom module involves:

1. Writing Python Code in a Separate File

The programmer writes functions, variables, or classes in a Python file.

This file becomes the custom module.

2. Saving the File with .py Extension

The module must be saved as a Python file.

The filename becomes the module name.

3. Importing the Module

The module can then be imported into another Python program to use its contents.

After importing, functions and variables inside the module become accessible.